RAG Assignment: Baseline Retrieval vs. Re-Ranking

Language Models and Agents

Name: Paila Tejeswara Rao
Roll Number: 2025204022
Course: Language Models and Agents
Document: NITI Aayog National Strategy for Artificial Intelligence

GitHub Repository:
https://github.com/tejeswararao04/rag-baseline-vs-reranking

---

1. Objective and Experimental Setup

The objective of this assignment is to compare a baseline Retrieval-Augmented Generation (RAG) pipeline with a two-stage retrieval pipeline that uses a Cross-Encoder for re-ranking. The main aim is to check whether selecting more relevant passages before generation improves the quality of the final answer.

The document used was NITI Aayog's *National Strategy for Artificial Intelligence*, a 114-page policy document covering India's AI strategy, research, adoption, sectoral applications, and responsible AI. Text was extracted from 109 non-empty pages and divided into 362 chunks. Each chunk contains 1000 characters with an overlap of 150 characters. The same corpus and chunks were used for both pipelines to ensure a fair comparison.

The implementation was developed in Python using a Jupyter Notebook on Kaggle. The main components were:

- **Document processing:** PyPDF was used to extract text from the PDF.
- **Embedding model:** `all-MiniLM-L6-v2` generated 384-dimensional embeddings for the chunks.
- **Vector search:** FAISS with `IndexFlatIP` was used to retrieve the top 10 chunks using normalized vectors and inner product, equivalent to cosine similarity.
- **Re-ranking:** `cross-encoder/ms-marco-MiniLM-L-6-v2` scored the query and each of the 10 retrieved chunks together.
- **Answer generation:** Gemini 3.8 Flash was used for the main comparison. Flash-Lite was used for one late batch after the free-tier quota was reached, with the same prompt.

The `make_prompt` function instructs the LLM to answer only from the supplied context and return "not found" when the answer is unavailable in that context.

2. Architecture Comparison

2.1 Pipeline 1: Baseline Retrieval

In the baseline pipeline, the embedding model converts every document chunk into a vector during indexing. For a new question, the same model converts the query into a vector. FAISS compares the query vector with the stored chunk vectors and returns the top 10 results. The first three results are passed to the LLM as context.

Query → Query Embedding → FAISS Top 10 → Top 3 → LLM Answer

This approach is efficient because document embeddings are computed only once. However, the Bi-Encoder encodes the query and document chunks independently. It may therefore rank a relevant passage lower when its terminology differs from the query or when the question depends on a specific condition or negation.

2.2 Pipeline 2: Cross-Encoder Re-Ranking

The second pipeline first retrieves the same top 10 candidates using FAISS. The Cross-Encoder evaluates each query-chunk pair jointly and assigns a relevance score. The candidates are sorted using these scores, and the top three re-ranked chunks are supplied to the LLM.

Query → FAISS Top 10 → Cross-Encoder Re-Ranking → New Top 3 → LLM Answer

Both pipelines use the same `make_prompt` function and prompt instructions, only the selected context chunks differ. This keeps the answer-generation setup consistent and allows the effect of re-ranking to be examined.

Unlike the Bi-Encoder, the Cross-Encoder processes the query and passage together, allowing attention between their tokens. It can therefore assess whether a passage directly addresses the question rather than relying only on the similarity of independently generated embeddings.

3. Qualitative and Rank Shift Analysis

The following experiments examine three retrieval challenges: a keyword trap, negation, and general-versus-specific information. Rank changes indicate how the Cross-Encoder reordered the candidates originally returned by FAISS.

3.1 Case 1: Keyword Trap - COREs Funding Amount

Query: "COREs funding amount per centre"

The relevant passage states that the financial component for Centres of Research Excellence (COREs) could be INR 50 crore to INR 100 crore per centre. The baseline top three did not contain the required passage.

Table 1: Rank changes for the COREs funding query

Chunk ID	Page	Old Rank	New Rank
p55_172	55	4	1
p53_165	53	8	2
p56_176	56	2	3

Answer 1 (baseline): "Not found."

Answer 2 (re-ranked): The model identified the proposed funding range as INR 50 crore to INR 100 crore per CORE.

This is a keyword-trap case because the relevant passage was not ranked highly enough by the baseline retriever. The critical chunk moved from rank 4 to rank 1, bringing it into the top three supplied to the LLM. The baseline avoided inventing a number but failed to answer a question whose information existed in the document.

3.2 Case 2: Negation - Data Ecosystems and Skilled Workforce

Query: "Why can AI adoption not succeed without data ecosystems and skilled workforce?"

This question involves a negative condition and two related requirements. The baseline ranked relevant information too low, so the final answer did not contain enough evidence.

Table 2: Rank changes for the data and workforce query

Chunk ID	Page	Old Rank	New Rank
p7_15	7	8	1
p71_231	71	3	2
p109_347	109	5	3

Answer 1 (baseline): "Not found."

Answer 2 (re-ranked): The model discussed the importance of high-quality data, the shortage of AI professionals, gaps in advanced expertise, and the need for reskilling.

This is a negation case because the query uses "not" and "without", making the relationship between the requirements important. The chunk originally ranked eighth moved to first, and another relevant chunk moved from fifth to third. Re-ranking made the required evidence available to the LLM and improved the answer.

3.3 Case 3: General vs. Specific - AI for All

Query: "What barriers need to be addressed to achieve the goals of AI for All?"

The baseline answer contained eight points, including repeated references to high resource costs, low awareness, and privacy concerns. After re-ranking, the answer was more concise and organized around four main barriers: lack of expertise, absence of data ecosystems, high resource cost and low awareness, and privacy and anonymization.

The relevant chunks moved from ranks 4 and 7 to ranks 2 and 3. This is a general-versus-specific case: the question asks for a broad summary, but the answer should focus on the main barriers rather than combine loosely related passages into a long list.

Since the document is a policy strategy rather than a rulebook with explicit penalties, this case evaluates relevance, coverage, and duplication instead of penalty extraction. Re-ranking improved the organization of the answer and reduced repetition.

3.4 Observed Limitation

Re-ranking did not improve every query. In one health-data privacy experiment, the baseline returned an answer discussing consent and data protection, while the re-ranked pipeline returned "Not found." Although the Cross-Encoder promoted passages about privacy safeguards, those passages did not provide sufficient context for the generated answer under the fixed prompt.

This shows that a higher-ranked passage is not automatically sufficient for answering a question. Retrieval quality, context selection, and the LLM's use of the selected evidence all affect the final result.

4. Latency Profiling

Latency was measured on Kaggle using five runs for the query "COREs funding amount per centre." The collection contained 362 chunks. Retrieval-only measurements and end-to-end measurements are shown below.

Table 3: Measured latency of the two pipelines

Stage	Time (seconds)
Baseline FAISS Top 10 retrieval	0.0081
Cross-Encoder re-ranking of 10 candidates	0.0382
Total retrieval with re-ranking	0.0463
Pipeline 1: Baseline with LLM	0.377
Pipeline 2: Re-ranking with LLM	0.467

Re-ranking increased retrieval time from approximately 8.1 ms to 46.3 ms. The additional re-ranking cost was about 38.2 ms. For the measured full-pipeline runs, the baseline took 0.377 seconds, compared with 0.467 seconds for the re-ranked pipeline.

Thus, re-ranking added approximately 90 ms to the measured end-to-end latency. For this collection of 362 chunks, the additional overhead was small relative to the improvement observed in the funding and negation examples. These measurements are specific to the tested environment and query, and should not be treated as guaranteed latency figures for larger collections or different hardware.

5. Why Not Use a Cross-Encoder on the Entire Collection?

A Cross-Encoder must process the query and each candidate passage together. Its relevance scores cannot be precomputed for all future queries because they depend on the query itself.

For a large document collection, scoring every query-chunk pair would increase computation and latency substantially. By contrast, the Bi-Encoder can precompute document embeddings once and retrieve a small candidate set efficiently.

The implemented solution therefore uses a two-stage approach:

1. Use the Bi-Encoder and FAISS to retrieve the top 10 candidates.
2. Use the Cross-Encoder to score only these 10 query-chunk pairs.
3. Select the top three re-ranked passages.
4. Pass those passages to the LLM for answer generation.

This design balances retrieval speed with relevance. FAISS `IndexFlatIP` performs exact vector search in the current implementation, it is not an approximate index. For substantially larger collections, an approximate nearest-neighbour index could be considered to reduce retrieval cost further.

6. Conclusion

This assignment compared direct dense retrieval with a two-stage retrieval and re-ranking pipeline on the NITI Aayog AI strategy document. The experiments showed that the baseline can miss important evidence when a relevant chunk appears outside the top three. Cross-Encoder re-ranking promoted important passages for the COREs funding and data/workforce queries, improving the resulting answers.

The general-versus-specific example also showed that re-ranking can reduce duplication and improve answer organization. However, the privacy example confirms that re-ranking does not guarantee better answers in every case. The latency measurements showed a modest additional cost for re-ranking on this small corpus.

Overall, the two-stage approach provides a practical balance between efficient candidate retrieval and more detailed relevance scoring.

Tools used: Python, Jupyter Notebook, PyPDF, Sentence Transformers, FAISS, and the Gemini API.